Data Analytics report

Sep 6, 2026, 12:30 PM UTC

Visualec INT8 Quantization and Edge-Energy Report

Technical explanation of ESP-DL INT8 quantization, measured ESP32-S3 inference evidence, and the boundaries of Visualec energy-savings claims.

Technical summary

Visualec performs real INT8 pedestrian inference on physical device `visualec_esp32s3_1` using Espressif's `pedestrian-pico-bootstrap`; the planned YOLOv8n-distilled `visualec_pico.espdl` artifact has not yet been trained or deployed. The observed device sample shows approximately **126 ms per inference** (**8 FPS compute capacity**), **299 ms round-trip latency**, one current person detection, ten additional completed frames, and zero protocol errors.

INT8 represents model weights and intermediate activations with 8-bit integers instead of 32-bit floating-point values. Holding tensor count constant, scalar storage falls to **25% of FP32**. The 435,440-byte `.espdl` artifact uses **20.76%** of its 2 MiB model partition; the 2,865,264-byte application uses **91.08%** of its 3 MiB OTA slot.

The device inference target is met (8 FPS versus at least 7 FPS, and 126 ms versus at most 140 ms), but the observed 299 ms round trip exceeds the 200 ms p95 acceptance target. No board-level joules-per-inference or building-level savings have been measured with a power meter, so configured wattage arithmetic is not presented as physical energy evidence.

ESP inference meets its compute gate; end-to-end latency still needs work

The physical ESP32-S3 sample completed ten frames while frame IDs advanced by ten, confirming continuous result flow for the observed interval. Device compute is inside both firmware acceptance thresholds: **8 FPS** is above the 7 FPS floor and **126 ms** is below the 140 ms ceiling. Round-trip latency is approximately **299 ms**, about 99 ms above the 200 ms p95 target, so network transfer, queueing, and backend handling remain the main performance gap.

Source: Visualec live system-status snapshot

Table: Visualec system-status API

INT8 reduces scalar storage and data movement by four times

The chart compares numeric representation only: it holds the number of parameters and activations constant. INT8 needs one byte per scalar versus four bytes for FP32, which makes weights and activations easier to fit in flash, PSRAM, caches, and working buffers. The implication is less data movement and integer-friendly execution; it does **not** guarantee a fourfold reduction in latency or electrical power because operator support, memory access, preprocessing, Wi-Fi, and control tasks remain.

Source: Espressif ESP-DL quantization documentation

Table: ESP-DL quantization schemes

The deployed model is quantized, but it is the bootstrap detector

The running device identifies its provider as `ESP-DL INT8` and its model as `pedestrian-pico-bootstrap`. Its compiled input is **224 x 224 x 3**, with a 288 x 160 RGB565 little-endian transport frame. Runtime settings include a dashboard-controlled confidence threshold (observed configuration: **0.45**), **0.50 NMS**, and **top-K 10**. The built `.espdl` model

occupies **435,440 bytes**, while the complete application binary occupies **2,865,264 bytes**. These file sizes cannot be converted directly into a parameter count because an `.espdl` artifact also contains graph structure, quantization metadata, and tensor information. The custom `visualec_pico.espdl` remains a planned output of the real-C270 training pipeline.

Source: Visualec edge firmware and model pipeline

Tables: Visualec ESP-IDF configuration, Visualec ESP32-S3 firmware runtime, Visualec-Pico ESP-DL model wrapper, Visualec real-frame model pipeline

Deployed model and platform parameters

Source: Visualec edge firmware and model pipeline

Tables: Visualec ESP-IDF configuration, Visualec ESP32-S3 firmware runtime, Visualec-Pico ESP-DL model wrapper, Visualec real-frame model pipeline

Parameter	Configured value	Evidence
Confidence threshold	0.45 dashboard configuration	backend configuration; applied by firmware
Device	ESP32-S3 N16R8, dual-core 240 MHz	firmware target and sdkconfig
External memory	8 MiB OPI PSRAM at 80 MHz	sdkconfig
Firmware SHA-256	0307A6F107CA349B809D8202A79050956BC80E5F3D67B13A032BA27087D3EE6F	built artifact hash
Model	pedestrian-pico-bootstrap / PICO_S8_V1	compiled component and device hello
Model input	224 x 224 x 3	bootstrap model metadata
Model SHA-256	9A728500A8E7114318FA15CE06788F9C8F501F360C40A78D124AB7E3273FAF95	built artifact hash
NMS / top-K	0.50 / 10	detector construction
Preprocessing	mean [0,0,0]; divisor [1,1,1]	ESP-DL wrapper
Quantization	w8a8 signed INT8, symmetric power-of-two per tensor	ESP-DL documentation
Runtime	ESP-DL 3.3.11; pedestrian_detect 0.3.2	managed component lock
Wire frame	288 x 160 RGB565LE; 92,188-byte packet	backend protocol and 28-byte header

Live Visualec edge-inference evidence

Source: Visualec live system-status snapshot

Table: Visualec system-status API

Device	Provider	Inference	Compute capacity	Round trip	People	Protocol errors	Frame IDs	Completed
visualec_esp32s3_1	ESP-DL INT8	126ms	8FPS	299ms	1	0	237 to 247	234 to 244

Model storage has ample headroom; the OTA application slot is tight

The model consumes only **20.76%** of its dedicated 2 MiB partition, leaving about **1.58 MiB** free. The application binary consumes **91.08%** of each 3 MiB OTA slot, leaving about **274 KiB**, which is the most immediate firmware-growth constraint. Compiled internal data RAM is **40.76%** used. The last retained device metric reported about **4.37 MiB free PSRAM**; because the edge link later disconnected, that runtime memory figure is retained rather than a fresh live reading.

Source: Visualec edge firmware and model pipeline

Tables: Visualec ESP-IDF configuration, Visualec ESP32-S3 firmware runtime, Visualec-Pico ESP-DL model wrapper, Visualec real-frame model pipeline

Exact storage and memory headroom

Source: Visualec edge firmware and model pipeline

Tables: Visualec ESP-IDF configuration, Visualec ESP32-S3 firmware runtime, Visualec-Pico ESP-DL model wrapper, Visualec real-frame model pipeline

Resource	Capacity	Used	Free	Used percent
OTA application slot	3.15Mbytes	2.87Mbytes	280.46Kbytes	91.08%
External PSRAM	8.39Mbytes	3.8Mbytes	4.58Mbytes	45.34%
Internal DIRAM	341.76Kbytes	139.31Kbytes	202.45Kbytes	40.76%
Model partition	2.1Mbytes	435.44Kbytes	1.66Mbytes	20.76%

Calibration maps real-valued tensors into signed 8-bit ranges

For each ESP32-S3 tensor, ESP-DL's current strategy uses symmetric, power-of-two, per-tensor quantization. A floating value `R` is mapped to `Q = clip(round(R / 2^Exp), -128, 127)` and reconstructed approximately as $\hat{R} = Q \times 2^{Exp}$. Calibration runs representative real C270 frames through the floating model to choose exponents that cover typical activation ranges.

The loss mechanism is explicit: values beyond the selected range saturate, while small differences can round to the same integer. Representative calibration frames are therefore essential. Visualec's intended pipeline uses PTQ first and accepts QAT only when the quantized student loses more than three percentage points against the floating student. No synthetic frames are permitted.

Source: Espressif ESP-DL quantization documentation

Table: ESP-DL quantization schemes

YOLOv8n teaches the compact model but does not run on the ESP

The intended production sequence is: real C270 session capture → YOLOv8n person-only pseudo-labeling → train a one-class ESPDet-Pico-derived student → export ONNX → calibrate and quantize for ESP32-S3 → package and hash the `.espdl` artifact → compare teacher, floating student, quantized host graph, and physical ESP outputs → flash only after quality gates pass.

This is knowledge distillation, not topology conversion. The ESP model learns from YOLOv8n's detections, but it uses a smaller architecture and ESP-DL-supported operators. Deploying stock YOLOv8n would exceed the practical latency and memory budget.

Runtime scheduling protects relay control from inference load

The ESP-IDF firmware separates relay control, inference, metrics, networking, and configuration work. Relay control runs at higher priority than inference; a one-frame newest-wins queue discards stale keyframes instead of building a backlog. Pixel packets use a versioned 28-byte header plus 92,160 bytes of RGB565 data. Occupancy and commands travel as JSON over a persistent WebSocket, and relay acknowledgements are issued only after the GPIO state is applied.

The N16R8 memory strategy places the model in flash-backed storage where supported, frame and model buffers in 8 MiB OPI PSRAM, and hot control state in internal SRAM. Active-low relay outputs use GPIO LOW for ON and GPIO HIGH for OFF. Automatic activation/deactivation defaults are 1 s and 10 s, while stale camera/link state forces safe-off after 30 s.

Source: Visualec edge firmware and model pipeline

Tables: Visualec ESP-IDF configuration, Visualec ESP32-S3 firmware runtime, Visualec-Pico ESP-DL model wrapper, Visualec real-frame model pipeline

Energy savings come from shorter integer work and relay-controlled off-time

Inference path. Smaller INT8 tensors reduce flash/PSRAM reads, cache pressure, and arithmetic work. The ESP32-S3 contributes a dual-core 240 MHz processor, a 128-bit data bus, and SIMD instructions suited to packed integer operations. The laptop still captures the C270 stream, performs motion selection and temporal tracking, and serves the dashboard; therefore the architecture removes laptop neural inference, not the laptop itself.

Control path. The ESP owns activation/deactivation timers and GPIO decisions. For a load `i`, physically saved energy is $P_i \times \texttt{verified_off_time}_i$. Across all zones, net system savings are the sum of avoided load energy plus avoided laptop inference energy, minus ESP and networking overhead. The present prototype defines three 9 W loads, so all three off represents 27 W of avoided load power while off; this is configuration arithmetic, not a watt-meter observation.

Source: Visualec live system-status snapshot

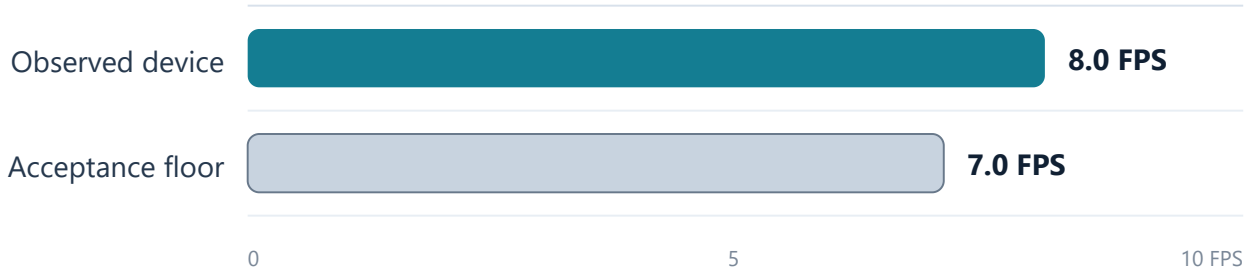
Table: Visualec system-status API

Performance at a glance

Physical ESP32-S3 verification sample; acceptance thresholds shown as engineering references.

Inference throughput versus acceptance floor

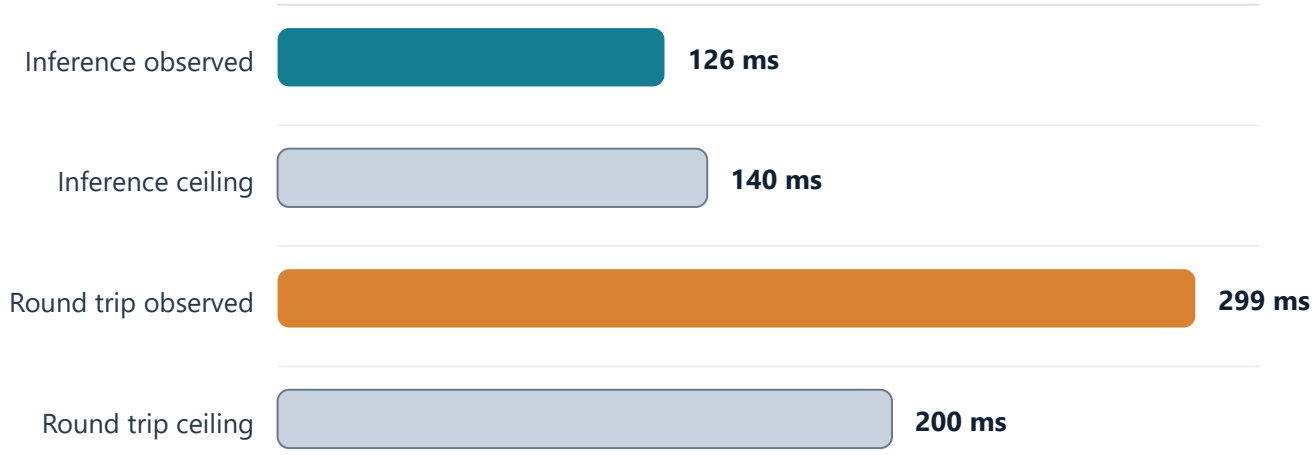
Frames per second; higher is better



Result: the observed device rate is 1 FPS above the minimum hardware gate.

Inference and round-trip latency versus ceilings

Milliseconds; lower is better; each observed value uses its own acceptance ceiling



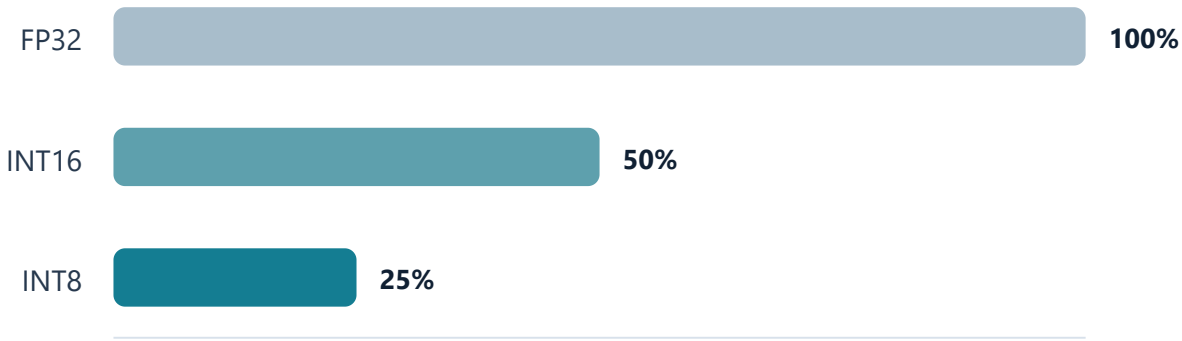
Result: device compute passes by 14 ms; observed round trip is 99 ms over the p95 target.

Quantization and memory footprint

Representation comparison holds tensor count constant; utilization values come from built artifacts and retained device metrics.

Storage per scalar by numeric format

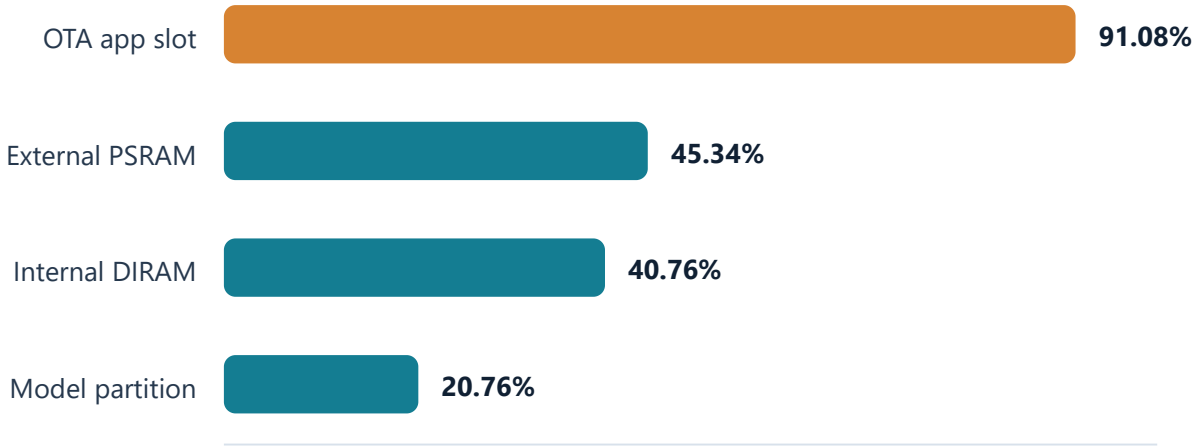
FP32 normalized to 100%; lower storage is better



INT8 stores each scalar in one byte versus four bytes for FP32. This is a 4x representation reduction, not a promise of 4x lower latency or power.

Memory and partition utilization

Percent used; compiled artifacts plus last retained runtime PSRAM reading



The model partition has 1,661,712 bytes free. The OTA application slot has only 280,464 bytes free and is the tighter growth constraint.

Current evidence proves execution, not electrical savings or final accuracy

The live snapshot proves that real frames reach the ESP, inference results return, and the detector is producing people counts. It does not measure joules per inference. Espressif's chip-level current tables are useful design context, but board regulators, PSRAM, Wi-Fi duty cycle, USB circuitry, relay coils, and the firmware's disabled Wi-Fi power saving change whole-board consumption.

The dashboard's energy figures are software-derived from configured relay wattages and state durations; they are not measurements from a current sensor. Likewise, the custom model has no real dataset, trained artifact, teacher-agreement report, or hardware recall evidence in the repository, so its energy or accuracy cannot yet be compared with the bootstrap model.

Measure energy and accuracy before claiming savings

1. Insert a USB power meter or calibrated shunt between the supply and ESP32-S3 board; record voltage/current at idle, connected/no inference, and sustained high-motion inference.
2. Record inference start/end markers and calculate `joules per inference = integral(V × I dt) / completed inferences` over at least 1,000 real frames.
3. Measure laptop wall power with local YOLO inference enabled and disabled while keeping capture, tracking, and dashboard activity identical.
4. Meter each controlled load or use verified nameplate power, then calculate savings from physical relay state durations.
5. Capture separate real C270 train, calibration, and held-out sessions; train and quantize the custom student and require the existing recall/accuracy gates.
6. Report total system energy as camera + laptop + ESP board + relay modules + controlled loads, not ESP consumption alone.

Further questions

- What are the real wattages and duty cycles of the final appliances rather than the 9 W prototype loads?
- Will the laptop remain powered for other work, or should a later architecture also replace laptop capture/tracking?
- What energy-per-inference and person-recall trade-off is acceptable if INT16 or mixed precision improves difficult detections?
- Can Wi-Fi transmission be duty-cycled without violating the relay-response and stale-link safety requirements?